

Makefile, Static Library and Input Handling

Static Library

A static library is compiled code that can be stored separately from the main C program and linked when the final program is built. In our project, we used a static library so the formatting code did not have to be placed directly inside main.c. This helped keep the C portion of the project separated into modules and made the purpose of each file easier to understand.

The static library used in the project is libformatter.a. First, formatter.c is compiled into an object file called formatter.o. That object file is then placed into the static library. The library can then be linked with main.c when the executable is created.

```
gcc -c formatter.c -o formatter.o
ar rcs libformatter.a formatter.o
gcc main.c libformatter.a -o schedule_formatter.exe
```

Once the program is linked, main.c can use the formatting function stored in the library without having all of the formatting code inside the main program. This shows how a static library can be created, reused, and linked as part of a larger C project.

Input File Handling

The input files required for this project are intended to be stored locally on the device being used to run the program. We have the program access the files directly from the project folder. We felt this was the best option because getting the information from the Internet would add another layer of complexity and another possible point of failure when the program is being tested or demonstrated.

Keeping the files in the project folder also makes the project easier to move from one computer to another. The program can access the files from the same folder structure instead of making every new user go through the code and replace path names with paths from their own system. The two main input types are the enrollment/class schedule information and the syllabus/professor information that includes office-hour information. The Python processing step cleans and combines the information before it is passed to the C portion of the program.

Makefile and Build Process

Our Makefile automates the project down to a simple make run command. When make run is used, main.py runs first so the input information is updated before the rest of the project is built. This is useful because the user may have changed the schedule or syllabus information since the last time the program was run. main.py produces Combined_Schedule_Office_Hours.txt, which stores the cleaned class schedule and office-hour information in one file for the C program to use.

Next, the Makefile checks the C dependencies. If formatter.c or formatter.h has changed, formatter.o is recompiled. If formatter.o changes, libformatter.a is rebuilt. The Makefile then compiles main.c and links it with libformatter.a to create schedule_formatter.exe. This prevents parts of the C project from being rebuilt when nothing related to them has changed.

After the executable is built, it runs using Combined_Schedule_Office_Hours.txt and creates formatted_schedule.txt. The last step runs generate_word.py, which reads the formatted text and creates Final_Schedule.docx with the selected instructor's class schedule and office hours. By putting

these steps in the Makefile, the project can be built and run in the correct order without manually typing every command each time.

```
1  CC = gcc
2  AR = ar
3  CFLAGS = -Wall -Wextra
4
5  TARGET = schedule_formatter.exe
6  LIBRARY = libformatter.a
7  OBJECT = formatter.o
8
9  .PHONY: all final run clean update_schedule
10
11 all: final
12
13 $(OBJECT): formatter.c formatter.h
14     $(CC) $(CFLAGS) -c formatter.c -o $(OBJECT)
15
16 $(LIBRARY): $(OBJECT)
17     $(AR) rcs $(LIBRARY) $(OBJECT)
18
19 $(TARGET): main.c formatter.h $(LIBRARY)
20     $(CC) $(CFLAGS) main.c $(LIBRARY) -o $(TARGET)
21
22 update_schedule:
23     python main.py
24
25 formatted_schedule.txt: $(TARGET) update_schedule
26     ./$$(TARGET)
27
28 Final_Schedule.docx: formatted_schedule.txt generate_word.py
29     python generate_word.py
30
31 final: Final_Schedule.docx
32     @echo "Final schedule generated successfully."
33
34 run: final
35
```

Figure 1. Makefile used to automate Python processing, C compilation, static library creation, linking, and final document generation.

Code Breakdown

main.py

main.py handles the first part of the input processing. It reads the enrollment/class schedule information and the syllabus information, extracts the course and office-hour data that the project needs, cleans the values, and combines the results into Combined_Schedule_Office_Hours.txt. The program was also updated so the combined data can include information for more than one instructor when the source information contains multiple instructors.

main.c

main.c starts the C portion of the project. It identifies the combined input text file and the formatted output text file, then calls the formatting function that is stored in the static library. Keeping main.c focused on starting the program and connecting the input and output files keeps the C code from becoming one large file.

formatter.c

formatter.c contains the formatting logic used by the C portion of the program. It reads through the information created by main.py and organizes the class schedule and office-hour sections into a cleaner format. It also helps handle empty or missing values so the text passed to the final Python program is consistent. formatter.c is compiled into formatter.o and then stored inside libformatter.a.

formatter.h

formatter.h is the header file for the C formatting module. It contains the function declaration that allows main.c to call the formatting function from formatter.c. This separates the function interface from the actual implementation and is part of how the project is organized into modules.

generate_word.py

generate_word.py handles the final document generation. It reads formatted_schedule.txt and creates Final_Schedule.docx in the class-schedule and office-hours format expected for the project. The script can select an instructor and place that instructor's course schedule and office hours into the final document. It was also updated so the final output still works when the processed information contains multiple instructors or information collected from more than one syllabus.

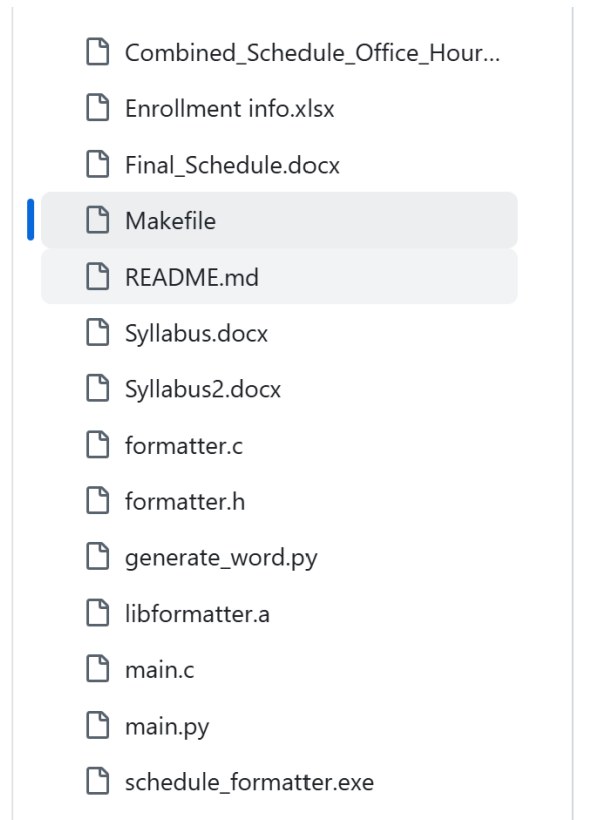


Figure 2. Project folder showing the C source files, header file, Python scripts, Makefile, static library, README, and generated output.

Note: The development folder screenshot includes an additional syllabus test file. The required project workflow is still described using the two main input sources: the enrollment/class schedule and the syllabus/professor information.

Final Output

The final output of the complete process is Final_Schedule.docx. The document contains the selected instructor's teaching schedule and office hours in a readable format. The sample schedule provided with the assignment was used as a guide for how the output should look, while the actual schedule and office-hour information comes from the project input files.

CLASS SCHEDULE

Instructor: Appolo Tankeh

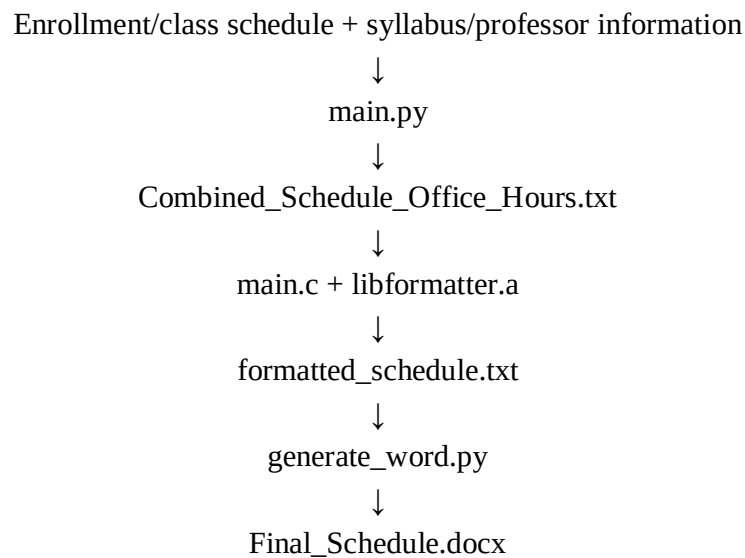
Course	Times	Location
CTEC 369 (Software Engineering)	Tuesday & Thursday 9:30 AM to 10:45 AM	108

OFFICE HOURS

NOTE: Instructors are not to be interrupted during class times.

Dr. Appolo Tankeh — Tue/Thursday: 10:45-1pm

Figure 3. Example of the generated class schedule and office-hours output for the selected instructor.



Conclusion

This part of the project showed how Python, C, a static library, and a Makefile can work together instead of having the entire project handled by one program. Python made it easier to work with the original files and create the final Word document, while C and the static library handled the formatting step. The Makefile connected the different parts and made sure they ran in the correct order. Overall, the project helped us understand how source files, header files, object files, static libraries, linking, and dependencies work together in a complete software project.